

CONIO_REFERENCE

Core Definition

`<conio.h>` ("console I/O") is a **non-standard** header for direct, unbuffered console access — reading keystrokes without Enter, positioning the cursor, and setting text colors. It appears in no C or C++ standard. Two substantially different implementations exist: the **Borland/Turbo C** version (the full screen-handling set) and the **Microsoft** version (a small subset, underscore-prefixed). GCC/Clang on Linux and macOS provide it at all only through third-party shims.

Treat this as a compatibility reference for legacy and coursework code. For new work, the portable equivalents are at the bottom of this file — and `<ncurses.h>/ncurses` is the standard answer for anything beyond a single keypress.

```
#CPP #CONIO #CONSOLE #GETCH #KBHIT #NON-STANDARD #DOS #BORLAND #MSVC #LEGACY
```

PORTABILITY MATRIX — read first

Function	Borland/Turbo C	MSVC (modern)	MinGW/GCC (Windows)	Linux/macOS
getch/getche	yes	_getch / _getche	yes (via MSVCRT)	NO
kbhit	yes	_kbhit	yes (via MSVCRT)	NO
putch	yes	_putch	yes	NO
ungetch	yes	_ungetch	yes	NO
cprintf/cputs	yes	_cprintf / _cputs	yes	NO
cscanf/cgets	yes	_cscanf_s/_cgets_s	partial	NO
clrscr	yes	NO	NO	NO
gotoxy	yes	NO	NO	NO
wherex/wherey	yes	NO	NO	NO
textcolor	yes	NO	NO	NO
textbackground	yes	NO	NO	NO
window	yes	NO	NO	NO
gettext/puttext	yes	NO	NO	NO
clreol/delline	yes	NO	NO	NO

*The screen-handling half of Borland's `conio.h` — colors, cursor positioning, windows — **does not exist in MSVC**. Code using `clrscr()` or `gotoxy()` is Turbo C-era and will not compile on a modern Microsoft toolchain without a shim. On Windows, the Console API (`<windows.h>`) provides those capabilities; everywhere else, `ncurses` does.*

COMPLETE CONIO QUICK REFERENCE

KEYBOARD INPUT — the portable-ish core (both implementations)

```
// =====  
// UNBUFFERED KEYBOARD INPUT — Target | Operation | Output  
// =====  
getch()           // none | Read key, NO echo, no Enter | Returns int keycode  
_getch()          //      | MSVC spelling of the same  
getche()          // none | Read key, WITH echo      | Returns int keycode  
_getche()         //      | MSVC spelling
```

```

kbhit() // none | Is a key waiting? | Returns non-zero if yes; does NOT
    block
_kbhit() // MSVC spelling
ungetch(ch) // int | Push a key back | Returns ch, or EOF on failure
_ungetch(ch) // MSVC spelling; ONE character of pushback only

// MSVC also provides _nolock variants that skip thread synchronization:
_getch_nolock() / _getche_nolock() / _putch_nolock() / _ungetch_nolock()

// =====
// CONSOLE OUTPUT
// =====
putch(ch) // int | Write char to console | Returns the char; bypasses stdout
_putchar(ch) // MSVC spelling
cputs(str) // char* | Write string to console | Returns non-negative; NO newline added
_cputs(str) // MSVC spelling
cprintf(fmt, ...) // format | printf to the console | Returns chars written
_cprintf(fmt, ...) // MSVC spelling
cscanf(fmt, ...) // format | scanf from the console | Returns fields assigned
_cscanf_s(fmt, ...) // MSVC bounds-checked form
cgets(buf) // buf | Read a line | buf[0] = max length IN, buf[1] = length
    OUT
_cgets_s(buf, size, &read) // MSVC bounds-checked form
// Wide variants (MSVC): _putwch, _cputws, _cwprintf, _cwsconf_s, _getwch, _getwche

```

SCREEN HANDLING — Borland / Turbo C only

```

// =====
// SCREEN CLEARING & LINE EDITING
// =====
clrscr() // none | Clear the active window | Cursor to (1,1)
clreol() // none | Clear to end of line | Cursor stays put
delline() // none | Delete the current line | Lines below move up
insline() // none | Insert a blank line | Lines below move down

// =====
// CURSOR POSITIONING – coordinates are 1-BASED, and (x, y) not (y, x)
// =====
gotoxy(x, y) // col,row | Move the cursor | 1-based, relative to the active window
wherex() // none | Current column | Returns int, 1-based
wherex() // none | Current row | Returns int, 1-based
_setcursortype(type) // int | Cursor shape | _NOCURSOR / _SOLIDCURSOR /
    _NORMALCURSOR

// =====
// TEXT ATTRIBUTES & COLOR
// =====
textcolor(color) // int | Foreground color | 0-15, or +BLINK
textbackground(color) // int | Background color | 0-7 only
textattr(attr) // int | Both at once | attr = fg | (bg << 4)
highvideo() // none | High intensity foreground
lowvideo() // none | Low intensity foreground
normvideo() // none | Restore the original attribute

// =====
// TEXT WINDOWS
// =====
window(left, top, right, bottom) // 4 ints | Define the active window | 1-based, inclusive
    // Subsequent output, clrscr and gotoxy are window-relative

// =====
// SCREEN MEMORY BLOCK OPERATIONS
// =====
gettext(l, t, r, b, buf) // rect+buf | Save a screen region | 2 bytes per cell: char + attribute
puttext(l, t, r, b, buf) // rect+buf | Restore a region | Returns non-zero on success
movetext(l, t, r, b, nl, nt) // rect+dest | Move a region | Returns non-zero on success

// =====
// SCREEN STATE

```

```
// =====
struct text_info {
    unsigned char winleft, wintop, winright, winbottom;
    unsigned char attribute;    // Current text attribute
    unsigned char normattr;     // Attribute at startup
    unsigned char currmode;     // Video mode (see enum text_modes)
    unsigned char screenheight, screenwidth;
    unsigned char curx, cury;   // Cursor position within the window
};
gettextinfo(&info)            // struct* | Fill in the above      | Returns void

// =====
// VIDEO MODE
// =====
textmode(mode)                // int | Set the text video mode
// enum text_modes: LASTMODE=-1, BW40=0, C40=1, BW80=2, C80=3, MONO=7, C4350=64

// =====
// GLOBALS
// =====
extern int directvideo;        // 1 = write straight to video RAM, 0 = via BIOS
extern int _wscroll;           // 1 = scroll the window on overflow (default), 0 = don't
```

COLOR CONSTANTS — Borland

```
// Foreground: 0-15. Background: 0-7 only (bit 3 is the blink bit).
BLACK      = 0    DARKGRAY   = 8
BLUE       = 1    LIGHTBLUE  = 9
GREEN      = 2    LIGHTGREEN  = 10
CYAN       = 3    LIGHTCYAN  = 11
RED        = 4    LIGHTRED    = 12
MAGENTA    = 5    LIGHTMAGENTA = 13
BROWN     = 6    YELLOW      = 14
LIGHTGRAY  = 7    WHITE       = 15

BLINK      = 128    // OR into a foreground color to blink

// textattr packs both: attr = foreground | (background << 4)
textattr(YELLOW | (BLUE << 4)); // Yellow on blue
```

KEY CODES

```
// =====
// ORDINARY KEYS – getch() returns the ASCII value directly
// =====
'a'-'z' 'A'-'Z' '0'-'9'      // Their ASCII codes
13 // Enter (CR – note: NOT \0)
27 // Esc
8  // Backspace
9  // Tab
32 // Space
3  // Ctrl+C (when not intercepted)
26 // Ctrl+Z

// =====
// EXTENDED KEYS – TWO getch() calls are required
// =====
// The first call returns 0 or 0xE0 (224); the SECOND returns the scan code.
// Which prefix you get depends on the key and the implementation, so test both.

int c = getch();
if (c == 0 || c == 0xE0) {
    c = getch(); // Now c holds the scan code below
}
```

```
// Scan codes after the prefix:
72 // Up arrow      80 // Down arrow
75 // Left arrow   77 // Right arrow
71 // Home        79 // End
73 // Page Up     81 // Page Down
82 // Insert      83 // Delete
59 // F1          60 // F2      61 // F3      62 // F4
63 // F5          64 // F6      65 // F7      66 // F8
67 // F9          68 // F10     133 // F11     134 // F12
```

Failing to consume the second byte is the classic `conio` bug: one arrow-key press then appears to the program as two keystrokes, and a menu jumps two entries at a time.

COMMON PATTERNS & EXAMPLES

Press Any Key

```
#include <conio.h>
#include <iostream>

int main() {
    std::cout << "Press any key to continue...";
    _getch();           // No Enter needed, nothing echoed
    std::cout << '\n';
}
```

Arrow-Key Menu

```
#include <conio.h>
#include <iostream>
#include <vector>
#include <string>

int menu(const std::vector<std::string>& items) {
    int sel = 0;
    while (true) {
        system("cls"); // Windows; see portability note
        for (std::size_t i = 0; i < items.size(); ++i)
            std::cout << (i == static_cast<std::size_t>(sel) ? " > " : " ")
                      << items[i] << '\n';

        int c = _getch();
        if (c == 0 || c == 0xE0) { // Extended key – read the scan code
            c = _getch();
            if (c == 72 && sel > 0) --sel; // Up
            else if (c == 80 && sel + 1 < (int)items.size()) ++sel; // Down
        } else if (c == 13) { // Enter
            return sel;
        } else if (c == 27) { // Esc
            return -1;
        }
    }
}
```

Non-Blocking Input Loop

```
#include <conio.h>
#include <iostream>

int main() {
    while (true) {
        updateGame();           // Keep running regardless of input

        if (_kbhit()) {         // Only true when a key is actually waiting
            int c = _getch();
            if (c == 27) break;   // Esc quits
            handleKey(c);
        }
    }
}
```

`_kbhit()` is what makes a game loop possible — `_getch()` alone would block until a key arrives.

Password Entry

```
#include <conio.h>
#include <iostream>
#include <string>

std::string readPassword(const char* prompt) {
    std::cout << prompt;
    std::string pw;
    int c;
    while ((c = _getch()) != 13) { // Until Enter
        if (c == 8) {             // Backspace
            if (!pw.empty()) {
                pw.pop_back();
                std::cout << "\b \b"; // Erase the asterisk on screen
            }
        } else if (c == 27) {     // Esc – abandon
            return {};
        } else if (c >= 32 && c < 127) { // Printable only
            pw += static_cast<char>(c);
            std::cout << '*';
        }
        // Extended keys arrive as 0/0xE0 + code; both are ignored here,
        // but the second byte must still be consumed:
        if (c == 0 || c == 0xE0) _getch();
    }
    std::cout << '\n';
    return pw;
}
```

Colored Text — Borland

```
#include <conio.h>

int main() {
    textbackground(BLUE);
    textcolor(YELLOW);
    clrscr(); // Fills the screen with the current background

    gotoxy(10, 5);
    cprintf("Warning"); // cprintf honors textcolor; printf does NOT

    textcolor(LIGHTGRAY);
    textbackground(BLACK);
    gotoxy(1, 24);
    cprintf("Press a key...");
}
```

```
    getch();
}
```

Only `cprintf`/`cputs`/`putch` respect the `conio` text attributes. Mixing `printf` or `std::cout` into colored output produces uncolored text, because those go through the standard streams instead.

Saving and Restoring a Screen Region — Borland

```
#include <conio.h>

void popup(const char* msg) {
    char saved[80 * 10 * 2];           // 2 bytes per cell: char + attribute
    gettext(20, 8, 59, 17, saved);     // Save what's underneath

    window(20, 8, 59, 17);
    textbackground(BLUE);
    textcolor(WHITE);
    clrscr();
    gotoxy(2, 2);
    cprintf("%s", msg);
    getch();

    window(1, 1, 80, 25);              // Restore the full-screen window
    puttext(20, 8, 59, 17, saved);     // Put the old content back
}
```

PORTABLE REPLACEMENTS

`getch` and `kbhit` Without `conio.h`

```
// Works on Windows (via conio) and POSIX (via termios).
#if defined(_WIN32)
    #include <conio.h>
    inline int portable_getch() { return _getch(); }
    inline bool portable_kbhit() { return _kbhit() != 0; }
#else
    #include <termios.h>
    #include <unistd.h>
    #include <sys/select.h>

    inline int portable_getch() {
        struct termios oldt;
        tcgetattr(STDIN_FILENO, &oldt);
        struct termios newt = oldt;
        newt.c_lflag &= ~(ICANON | ECHO);    // Raw-ish: no line buffering, no echo
        newt.c_cc[VMIN] = 1;
        newt.c_cc[VTIME] = 0;
        tcsetattr(STDIN_FILENO, TCSANOW, &newt);

        int c = getchar();

        tcsetattr(STDIN_FILENO, TCSANOW, &oldt); // ALWAYS restore
        return c;
    }

    inline bool portable_kbhit() {
        struct termios oldt;
        tcgetattr(STDIN_FILENO, &oldt);
        struct termios newt = oldt;
        newt.c_lflag &= ~(ICANON | ECHO);
        tcsetattr(STDIN_FILENO, TCSANOW, &newt);
    }
}
```

```

    timeval tv{0, 0};
    fd_set fds;
    FD_ZERO(&fds);
    FD_SET(STDIN_FILENO, &fds);
    int ready = select(STDIN_FILENO + 1, &fds, nullptr, nullptr, &tv);

    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
    return ready > 0;
}
#endif

```

Restoring the terminal is not optional. A program that exits — or crashes — with `ICANON/ECHO` cleared leaves the user's shell with no echo and no line editing. Wrap the `termios` change in an RAII guard so the destructor restores it on every path, including exceptions.

RAII Terminal Guard (POSIX)

```

#include <termios.h>
#include <unistd.h>

class RawMode {
    termios old_{};
    bool active_ = false;
public:
    RawMode() {
        if (tcgetattr(STDIN_FILENO, &old_) == 0) {
            termios raw = old_;
            raw.c_lflag &= ~(ICANON | ECHO);
            raw.c_cc[VMIN] = 1; raw.c_cc[VTIME] = 0;
            active_ = (tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw) == 0);
        }
    }
    ~RawMode() { if (active_) tcsetattr(STDIN_FILENO, TCSAFLUSH, &old_); }
    RawMode(const RawMode&) = delete;
    RawMode& operator=(const RawMode&) = delete;
};

```

Screen Handling — The Real Answers

conio.h (Borland)	ncurses / PDCurses	Windows Console API
<code>clrscr()</code>	<code>erase()</code> / <code>clear()</code>	<code>FillConsoleOutputCharacter</code>
<code>gotoxy(x, y)</code>	<code>move(y, x)</code> // y FIRST	<code>SetConsoleCursorPosition</code>
<code>wherex()</code> / <code>wherey()</code>	<code>getyx(win, y, x)</code>	<code>GetConsoleScreenBufferInfo</code>
<code>textcolor(c)</code>	<code>attron(COLOR_PAIR(n))</code>	<code>SetConsoleTextAttribute</code>
<code>textbackground(c)</code>	<code>init_pair(n, fg, bg)</code>	<code>SetConsoleTextAttribute</code>
<code>window(l,t,r,b)</code>	<code>newwin(h, w, y, x)</code>	(no direct equivalent)
<code>gettext</code> / <code>puttext</code>	<code>overwrite()</code> / <code>copywin()</code>	<code>ReadConsoleOutput</code> / <code>Write...</code>
<code>getch()</code>	<code>getch()</code> // same name	<code>ReadConsoleInput</code>
<code>kbhit()</code>	<code>nodelay(win, TRUE)</code>	<code>PeekConsoleInput</code>
<code>cprintf()</code>	<code>printw()</code> / <code>wprintw()</code>	<code>WriteConsole</code>
<code>_setcursortype()</code>	<code>curs_set(0/1/2)</code>	<code>SetConsoleCursorInfo</code>

*ncurses on Unix and **PDCurses** on Windows share an API, so a curses program is portable across both. That is the migration path for any `conio.h` code that outgrows a single `getch()`. See `CURSES_REFERENCE` for the full curses API.*

ANSI Escape Sequences — a lightweight middle ground

```
#include <cstdio>
// Supported by Linux/macOS terminals and by Windows 10+ consoles once
// ENABLE_VIRTUAL_TERMINAL_PROCESSING is set via SetConsoleMode.
#define CLS "\033[2J\033[H"
#define GOTOXY(x,y) "\033[" #y ";" #x "H"
#define FG_RED "\033[31m"
#define FG_YELLOW "\033[33m"
#define BG_BLUE "\033[44m"
#define RESET "\033[0m"

std::printf(CLS FG_YELLOW BG_BLUE " Warning " RESET "\n");
```

IMPORTANT CONCEPTS

This Header Is Not Standard — At All

`conio.h` appears in no ISO C or C++ standard, and there is no specification defining its behavior. Every property below — return values, key codes, coordinate origin — is implementation-defined. Code depending on it is not portable, and often not portable even between two Windows compilers.

The Two Dialects Are Genuinely Different

Borland's version came from the Turbo C DOS era and manipulated video memory directly. Microsoft's version is a thin wrapper over console input and provides no screen handling whatsoever. A textbook or course using `clrscr()` and `gotoxy()` is teaching the Borland dialect, which no current Microsoft compiler supports.

`getch()` Bypasses the Standard Streams

It reads the console directly, not `stdin`. Mixing it with `std::cin` or `scanf` gives confusing results, because the two draw from different places and buffer independently. It also means `getch()` does not work when input is redirected from a file or a pipe — a program that pauses on `getch()` will hang or skip depending on the implementation.

Coordinates Are 1-Based and `(x, y)`

`gotoxy(x, y)` takes column first, row second, both starting at 1. Curses uses `move(y, x)` — row first, column second, both starting at 0. Porting between them requires swapping the arguments *and* adjusting the origin; getting one but not the other is a frequent porting bug.

Extended Keys Are Two Reads

An arrow or function key produces a `0` or `0xE0` prefix followed by a scan code. Both bytes must be consumed. Which prefix appears is not consistent across implementations and key types, so always test for both.

`system("cls")` Is Not a Clear-Screen Function

It spawns a shell, which is slow, flickers, is `cls` on Windows but `clear` on Unix, and executes whatever program happens to be named `cls` on the user's `PATH`. It is convenient in coursework and inappropriate in anything shipped. ANSI escapes or curses are the real answers.

The Terminal Must Be Restored

On POSIX, putting the terminal into non-canonical mode is a change to shared state that outlives the process. Any path out of the program — `return`, `exit`, an exception, a signal — must restore it, which means RAII, and

ideally a signal handler too. `conio.h` on Windows has no equivalent hazard, which is one reason ported code so often gets this wrong.

What to Use Instead

For a single "press any key", a `#if defined(_WIN32)` shim like the one above is proportionate. For anything with a cursor, colors, windows, or a redrawn screen, use `curses` — `ncurses` on Unix, `PDCurses` on Windows, same API on both. For a modern cross-platform TUI in C++, `FTXUI` is a well-maintained option that needs no platform conditionals at all.

BEST PRACTICES

1. **Do not use `conio.h` in new cross-platform code** — it will not compile on Linux or macOS
2. **Isolate it behind your own interface** (`portable_getch`) so the platform detail lives in one file
3. **Always consume the second byte** of an extended key sequence
4. **Never mix `getch()` with `std::cin`** — different input paths, different buffers
5. **Use `_getch` / `_kbhit`** (underscore forms) on MSVC — the unprefixed names are deprecated there
6. **Restore terminal state with `RAII`** in any POSIX implementation
7. **Avoid `system("cls")`** — slow, non-portable, and a security hazard
8. **Do not assume `getch()` works with redirected input**
9. **Reach for `curses`** the moment you need more than one keypress at a time
10. **Remember Borland `gotoxy` is 1-based (`x, y`)** while `curses move` is 0-based (`y, x`)
11. **Only `cprintf` / `cputs` / `putch` honor `textcolor`** — standard output functions ignore it

RELATED HEADERS

```
#include <conio.h>      // Non-standard; Windows/DOS toolchains only
#include <curses.h>      // ncurses (Unix) / PDCurses (Windows) — the portable answer
#include <termios.h>     // POSIX terminal control — raw mode, echo
#include <unistd.h>      // POSIX read/write/STDIN_FILENO
#include <sys/select.h>  // POSIX select() — non-blocking input check
#include <windows.h>     // Windows Console API — colors, cursor, screen buffer
#include <cstdio>         // Standard, portable console I/O
#include <iostream>      // Standard C++ streams
```

EXTERNAL RESOURCES

- **Microsoft `_getch`**: <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/getch-getwch>
- **Microsoft console functions**: <https://learn.microsoft.com/en-us/cpp/c-runtime-library/console-and-port-i-o>
- **Windows Console API**: <https://learn.microsoft.com/en-us/windows/console/console-functions>
- **ncurses HOWTO**: <https://tldp.org/HOWTO/NCURSES-Programming-HOWTO/>
- **PDCurses (Windows curses)**: <https://pdcurses.org/>
- **FTXUI (modern C++ TUI)**: <https://github.com/ArthurSonzogni/FTXUI>

Standard: None — non-standard header. Borland/Turbo C and Microsoft dialects differ substantially. **Platform:** Windows (MSVC subset, MinGW), DOS (Borland full set). Not available on Linux or macOS. **Last Updated:**

September 2026